

Compiler Design Project Report

Mini Compiler for a Domain-Specific Language for Educational Assessments

Submitted by

Deekshitha S K 1RVU22CSE047

Smaran R Bharadwaj 1RVU22CSE157

School of Computer Science

RV University

Submitted to

Prof. Uma Shankari B

Asst. Professor

School of Computer Science

RV University

15-04-2025

Certificate

Certified that Deekshitha S K (1RVU22CSE047) and Smaran R Bharadwaj (1RVU22CSE157) have carried out the Project work presented in this report entitled

"Mini Compiler for a Domain-Specific Language in Educational Assessments"

for the Compiler Design course in partial fulfillment of B.Tech.(Hons) in School of Computer Science and Engineering of the RV University, Bengaluru during the year 2025-2026

under my supervision. The report embodies results of original work and studies carried out by students themselves and the contents of the Project do not form the basis for the award of any other degree to the candidate or to anybody else.

Prof. Uma Shankari B

Designation:

Asst. Professor, SoCSE

Date: 15-04-2025

Abstract

This project focuses on designing and implementing a mini compiler for a DSL (domain specific language) targeted at high school teachers to generate assessments, using Lex and Yacc tools. The main objective is to parse a simple programming language, perform semantic analysis, and generate an abstract syntax tree (AST). By combining lexical analysis and parsing techniques, this project demonstrates a functional compiler component that validates syntax and semantics of the DSL code. This project showcases successful implementation of key compiler phases such as token generation (lexical phase), parsing, and AST creation (syntax and semantic analysis phase).

Compiler Design Project Report

1. Introduction (1 page)

Project Background and Relevance:

Compiler design is a fundamental aspect of computer science, enabling translation of high-level programming languages into machine code. Understanding compiler components like lexical analyzers, parsers, and semantic analyzers is crucial for language design and implementation.

Given the rise of auto grading and on-computer assessments, the DSL developed during the course of the project aims to provide a platform for high school educators who have a beginner's experience level in programming to automate their assessments. This project implements the front-end of a compiler, focusing on lexical, syntax and semantic analysis.

Objectives:

- Implement the first three phases of a compiler — lexical analysis, syntax analysis, and semantic analysis — for a domain-specific language (DSL) focused on developing educational assessments.
- Design and develop the DSL along with its grammar rules (aimed at MCQ/True and False questions).
- Use Lex to build a lexical analyzer that tokenizes the source code.
- Use Bison to create a parser that parses tokens based on defined grammar rules.
- Construct an Abstract Syntax Tree (AST) from parsed constructs.
- Perform semantic analysis to validate the correctness and consistency of expressions.

2. Methodology

Parser explanation (Algorithm Explanation)

The parser uses context-free grammar rules defined in `assessment.y` to interpret the structure of the input code. The Yacc grammar is associated with semantic actions in C, which build the AST or perform semantic checks during parsing. Lex generates tokens for keywords, operators, identifiers, and literals using patterns defined in `assessment.l`. The semantic analyzer (in `semantic.c`) traverses the syntax tree to enforce language rules.

Workflow:

Compiler Design Project Report

1. Lex scans source code and produces tokens.
2. Bison receives tokens and matches them against grammar rules.
3. On a valid match, associated C code creates AST nodes or checks semantics.
4. Errors are reported if unexpected tokens or semantic violations occur.

Specification needed (Language, tools, system specification):

- Language: C
- Tools: Lex, Yacc (Bison), GCC
- System Requirements:
 - Ubuntu 22.04 / WSL / MacOS 15+ for toolchain support
 - Memory: 4GB+
 - GCC compiler
 - Flex & Bison installed

3. Implementation

Important Code Snippets:

Lex Specification (assessment.l)

```
%%
"int"      { return INT; }
"return"   { return RETURN; }
[0-9]+     { yylval = atoi(yytext); return NUM; }
[a-zA-Z_][a-zA-Z0-9_]* { yylval = insert_symbol(yytext);
return ID; }
"+"       { return PLUS; }
\n        { /* ignore */ }
[ \t]     { /* ignore whitespace */ }
.         { printf("Unknown character: %s\n", yytext); }
%%
```

Yacc Specification (assessment.y)

```
%%
program:
    function_list
    ;

function_list:
    function_list function
    | function
    ;

function:
    INT ID '(' ')' '{' stmt_list RETURN expr ';' '}'
```

Compiler Design Project Report

```
    ;

stmt_list:
    stmt_list stmt
    | stmt
    ;

stmt:
    ID '=' expr ';'
    ;

expr:
    expr PLUS term
    | term
    ;

term:
    NUM
    | ID
    ;

%%
```

AST and Semantic Checks (semantic.c)

```
void semantic_check(ASTNode* node) {
    if (node == NULL) return;
    // Recursively check types and validity
    if (node->type == NODE_ASSIGN) {
        if (!is_declared(node->left->name))
            printf("Semantic Error: Undeclared variable\n", node->left->name);
    }
    semantic_check(node->left);
    semantic_check(node->right);
}
```

GitHub repo:

https://github.com/smaran-rvu/sem6_mini_compiler_assessment_DSL.git

Compiler Design Project Report

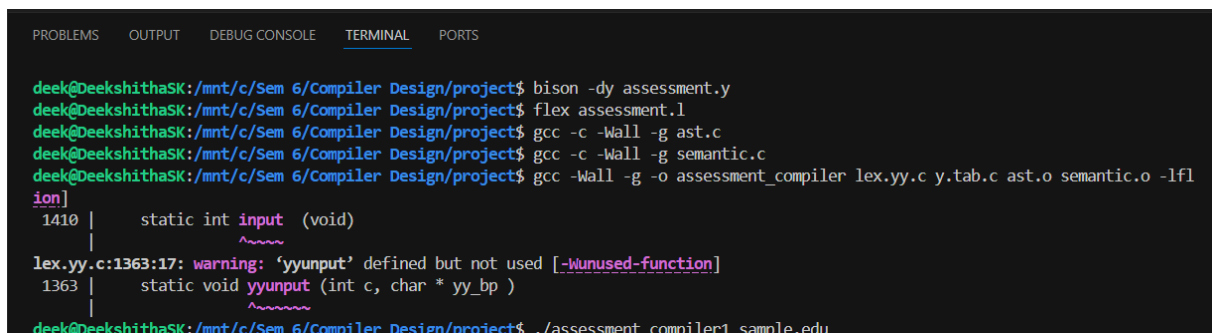
Results

Here are screenshots showing the execution and result of the compiler front-end:

1. Lex and Yacc Compilation

The following are the commands to develop the mini compiler from the lex and grammar files

```
$ bison -dy assessment.y (developing the tokens and syntax analyzer )
$ flex assessment.l (developing the lexical analyzer)
$ gcc -c -Wall -g ast.c (developing the Abstract Syntax Tree generator)
$ gcc -c -Wall -g semantic.c (developing the semantic analyzer)
$ gcc -Wall -g -o assessment_compiler lex.yy.c y.tab.c ast.o semantic.o -lfl
(the last argument is -lfl for linux systems)
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
deek@DeekshithaSK:/mnt/c/Sem 6/Compiler Design/project$ bison -dy assessment.y
deek@DeekshithaSK:/mnt/c/Sem 6/Compiler Design/project$ flex assessment.l
deek@DeekshithaSK:/mnt/c/Sem 6/Compiler Design/project$ gcc -c -Wall -g ast.c
deek@DeekshithaSK:/mnt/c/Sem 6/Compiler Design/project$ gcc -c -Wall -g semantic.c
deek@DeekshithaSK:/mnt/c/Sem 6/Compiler Design/project$ gcc -Wall -g -o assessment_compiler lex.yy.c y.tab.c ast.o semantic.o -lfl
ion]
1410 | static int input (void)
      | ~~~~~
lex.yy.c:1363:17: warning: 'yyunput' defined but not used [-Wunused-function]
1363 | static void yyunput (int c, char * yy_bp )
      | ~~~~~
deek@DeekshithaSK:/mnt/c/Sem 6/Compiler Design/project$ ./assessment_compiler sample.edu
```

Screenshot:

2. Output for Sample Input

The file `sample.edu` contains the sample input of the DSL that is given as input the mini compiler.

sample.edu:

```
ASSESSMENT "Biology Quiz" {

    SECTION "Cell Structure" {

        MC_QUESTION "Which organelle is responsible for
energy production?" {
            OPTION "Nucleus" INCORRECT
            OPTION "Mitochondria" CORRECT
            OPTION "Ribosome" INCORRECT
            OPTION "Golgi apparatus" INCORRECT

            FEEDBACK "Mitochondria are often called the
powerhouse of the cell."
        }
    }
}
```

Compiler Design Project Report

```
        TF_QUESTION "Prokaryotic cells contain a
nucleus." {
            ANSWER FALSE
            FEEDBACK "Prokaryotic cells do not have
membrane-bound organelles."
        }
    }

    SECTION "Genetics Basics" {

        TF_QUESTION "DNA stands for Deoxyribonucleic
Acid." {
            ANSWER TRUE
            FEEDBACK "DNA (Deoxyribonucleic Acid)
contains the genetic instructions for development and
functioning of living organisms."
        }

        MC_QUESTION "Which of the following is a
nucleotide base found in DNA?" {
            OPTION "Uracil" INCORRECT

            FEEDBACK "Adenine is one of the four
nucleotide bases in DNA, along with guanine, cytosine,
and thymine."
        }
    }

    IF_SCORE ABOVE 3 {
        SECTION "Advanced Topics" {
            MC_QUESTION "Which of the following processes
generates ATP?" {
                OPTION "Translation" INCORRECT
                OPTION "Cellular respiration" CORRECT
                OPTION "DNA replication" INCORRECT
                OPTION "Transcription" INCORRECT

                FEEDBACK "Cellular respiration is the
process that generates ATP, the energy currency of
cells."
            }
        }
    }
}
```


Compiler Design Project Report

Output after mini compiler has processed the file:

```
(base) smaranrb@Smarans-MacBook-Pro Project % ./assessment_compiler sample.edu
Parsing successful! Printing AST:
ASSESSMENT: Biology Quiz
SECTIONS
  SECTION: Cell Structure
  QUESTIONS
    MC_QUESTION: Which organelle is responsible for energy production?
    MC_QUESTION: options
      OPTION: Nucleus (INCORRECT)
      OPTION: Mitochondria (CORRECT)
      OPTION: Ribosome (INCORRECT)
      OPTION: Golgi apparatus (INCORRECT)
    FEEDBACK: Mitochondria are often called the powerhouse of the cell.
    TF_QUESTION: Prokaryotic cells contain a nucleus.
    ANSWER: FALSE
    FEEDBACK: Prokaryotic cells do not have membrane-bound organelles.
  SECTION: Genetics Basics
  QUESTIONS
    TF_QUESTION: DNA stands for Deoxyribonucleic Acid.
    ANSWER: TRUE
    FEEDBACK: DNA (Deoxyribonucleic Acid) contains the genetic instructions for development and functioning of living organisms.
    MC_QUESTION: Which of the following is a nucleotide base found in DNA?
    MC_QUESTION: options
      OPTION: Uracil (INCORRECT)
      FEEDBACK: Adenine is one of the four nucleotide bases in DNA, along with guanine, cytosine, and thymine.
  IF SCORE: ABOVE 3
  IF SCORE: ABOVE -1
  SECTION: Advanced Topics
  QUESTIONS
    MC_QUESTION: Which of the following processes generates ATP?
    MC_QUESTION: options
      OPTION: Translation (INCORRECT)
      OPTION: Cellular respiration (CORRECT)
      OPTION: DNA replication (INCORRECT)
      OPTION: Transcription (INCORRECT)
    FEEDBACK: Cellular respiration is the process that generates ATP, the energy currency of cells.

Performing semantic analysis...
Semantic Error: MC_QUESTION must have at least two options (in MC_QUESTION "Which of the following is a nucleotide base found in DNA?")
Semantic Error: MC_QUESTION must have exactly one correct option (in MC_QUESTION "Which of the following is a nucleotide base found in DNA?")
Semantic analysis failed! Please fix the errors.
(base) smaranrb@Smarans-MacBook-Pro Project %
```

Output after successful parsing through the Semantically verified Abstract Syntax tree Features:

- Root level: ASSESSMENT: Biology Quiz
- First level: SECTIONS (container for all sections)
- Second level: Individual SECTIONs like "Cell Structure", "Genetics Basics"
- Third level: QUESTIONS (container for questions in each section)
- Fourth level: Individual questions (MC_QUESTION and TF_QUESTION)

4. Conclusion & Future Enhancements

This project successfully demonstrates the key components of a compiler front-end using Bison and Yacc. It performs lexical analysis, parsing, and semantic validation of a simplified language. An abstract syntax tree was created for structured representation. The project helped understand compiler pipelines, grammar implementation, and error handling mechanisms.

Future enhancements to the project include:

- Extending DSL scope to also include question types like short answer/descriptive questions.
- Integrate a symbol table for tracking declarations and scopes.
- Add code generation for intermediate or assembly-like output.
- Develop a basic IDE interface for writing and testing code.